

Case Study: Search Engine Query Processing Using Binary Search

CSA0613-Design and Analysis of Algorithms

Course Outcome: CO3_AT2

Student Name: A. Bhavya(192424417)

Q3. Case Study: Search Engine Query Processing (Binary Search)

A search engine maintains a sorted index of keywords and must retrieve results quickly. Analyse how Binary Search improves search efficiency. Identify key requirements such as sorted data and fast retrieval. Apply divide and conquer principles to explain the process. Analyse time complexity and performance. Design a solution and justify why Binary Search is optimal in this context.

1. Problem Overview

Search engines maintain a large collection of keywords in a sorted order to retrieve information quickly. Binary Search is used to reduce the search time and improve efficiency. The algorithm divides the data into smaller parts until the required keyword is found.

2. Objectives of the Study

- Understand the role of Binary Search in search engines.
- Improve search efficiency.
- Retrieve results quickly from large datasets.
- Apply the divide-and-conquer technique.
- Analyze performance and complexity.

3. System Requirements

The following requirements are necessary for implementing Binary Search:

- Keywords must be stored in sorted order.
- Fast retrieval of data is required.
- Efficient memory utilization.
- Quick comparison of elements.

- Support for large datasets.

4. Working Principle of Binary Search

Binary Search follows the divide-and-conquer approach.

Steps:

1. Find the middle element.
2. Compare it with the search keyword.
3. If both are equal, return the result.
4. If the keyword is smaller, search the left half.
5. If the keyword is larger, search the right half.
6. Repeat the process until the element is found.

5. Algorithm Design:

BinarySearch(arr, low, high, key)

while low <= high

 mid = (low + high) / 2

 if arr[mid] == key

 return mid

 else if key < arr[mid]

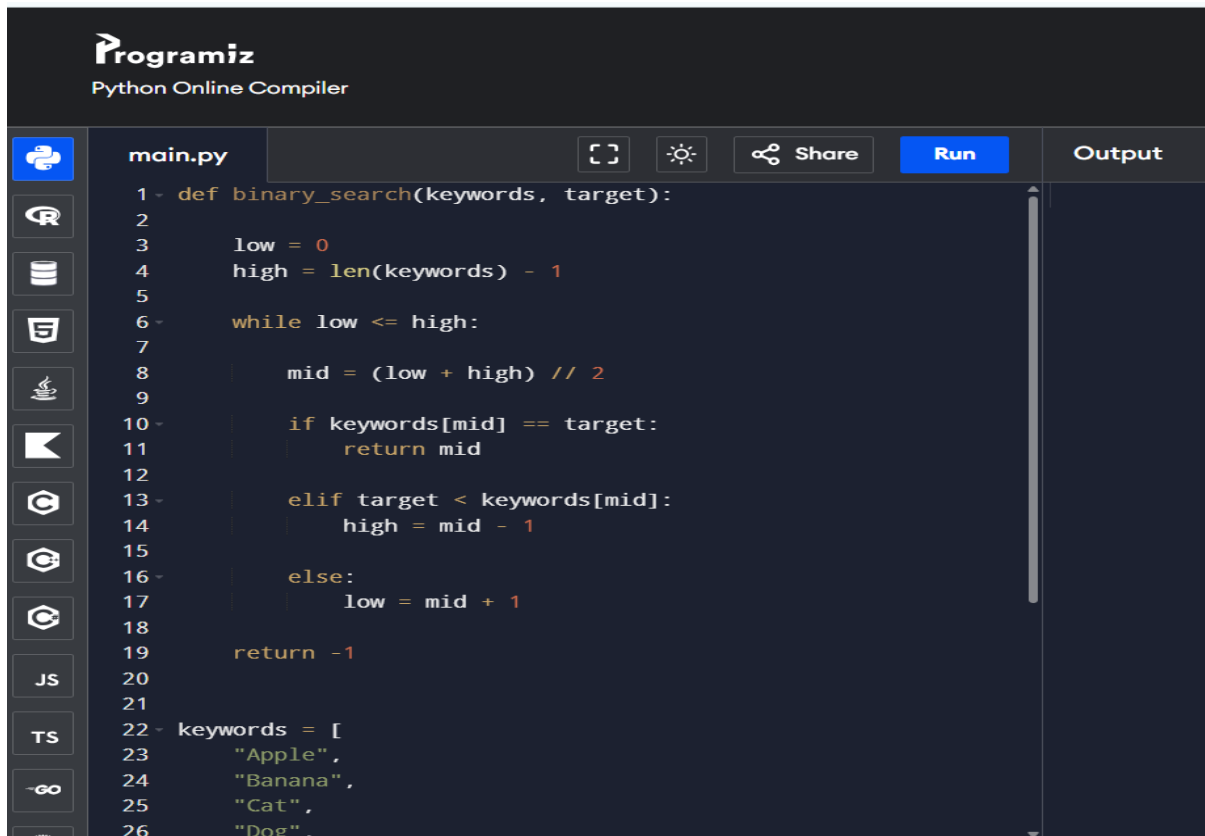
 high = mid - 1

 else

 low = mid + 1

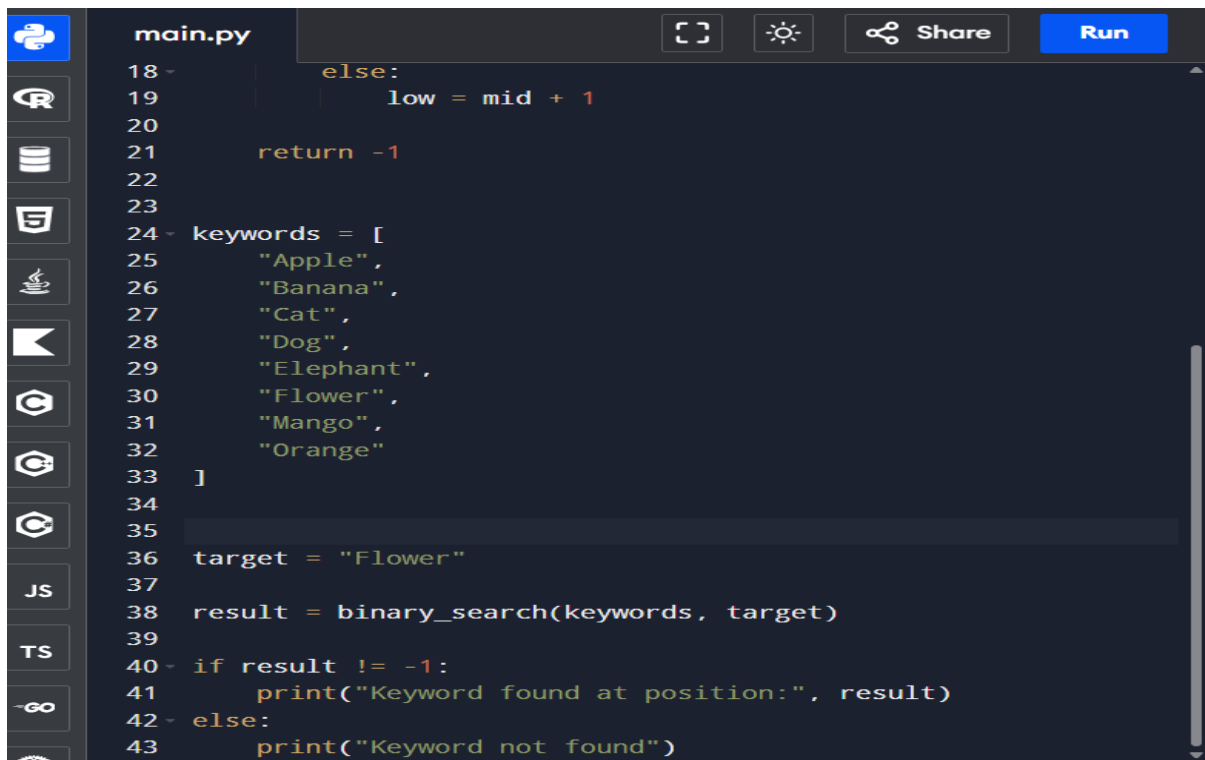
return -1

SOURCE CODE:



The screenshot shows the Programiz Python Online Compiler interface. The editor displays the first part of a binary search algorithm in a file named `main.py`. The code defines a `binary_search` function that takes `keywords` and `target` as arguments. It initializes `low` to 0 and `high` to `len(keywords) - 1`. A `while` loop runs as long as `low` is less than or equal to `high`. Inside the loop, `mid` is calculated as `(low + high) // 2`. If `keywords[mid]` equals `target`, it returns `mid`. If `target` is less than `keywords[mid]`, it updates `high` to `mid - 1`. Otherwise, it updates `low` to `mid + 1`. After the loop, it returns `-1`. Below the function definition, a list of keywords is defined: `keywords = ["Apple", "Banana", "Cat", "Dog"]`. The interface includes a left sidebar with various icons, a top bar with the Programiz logo and "Python Online Compiler" text, and buttons for "Share", "Run", and "Output".

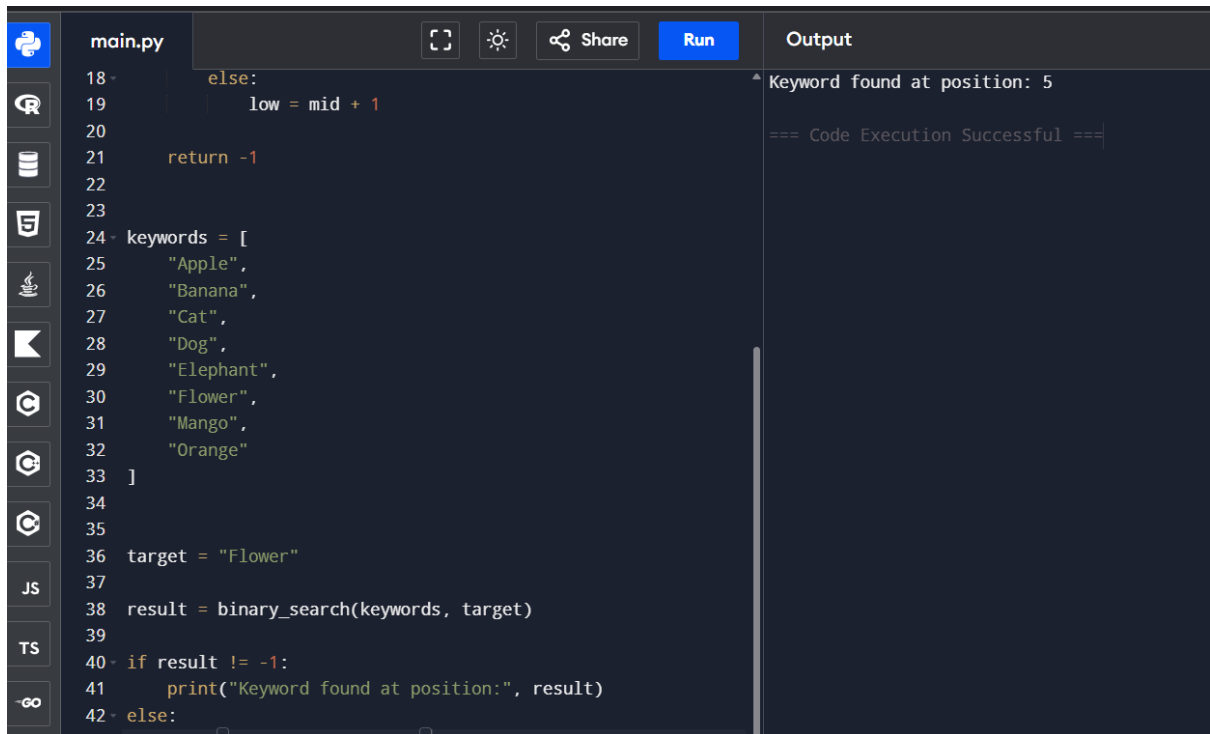
```
1 def binary_search(keywords, target):
2
3     low = 0
4     high = len(keywords) - 1
5
6     while low <= high:
7
8         mid = (low + high) // 2
9
10        if keywords[mid] == target:
11            return mid
12
13        elif target < keywords[mid]:
14            high = mid - 1
15
16        else:
17            low = mid + 1
18
19    return -1
20
21
22 keywords = [
23     "Apple",
24     "Banana",
25     "Cat",
26     "Dog",
```



The screenshot shows the second part of the binary search algorithm in the `main.py` file. It continues the `keywords` list with `"Elephant", "Flower", "Mango", "Orange"`. It then sets `target = "Flower"` and calls `result = binary_search(keywords, target)`. A conditional statement checks if `result` is not equal to `-1`. If true, it prints `"Keyword found at position:", result`. Otherwise, it prints `"Keyword not found"`. The interface is the same as the previous screenshot, showing the Programiz logo, "Python Online Compiler" text, and buttons for "Share", "Run", and "Output".

```
27     "Elephant",
28     "Flower",
29     "Mango",
30     "Orange"
31 ]
32
33
34
35
36 target = "Flower"
37
38 result = binary_search(keywords, target)
39
40 if result != -1:
41     print("Keyword found at position:", result)
42 else:
43     print("Keyword not found")
```

OUTPUT:



The screenshot shows a code editor with a file named `main.py`. The code implements a binary search algorithm. It defines a list of keywords: `["Apple", "Banana", "Cat", "Dog", "Elephant", "Flower", "Mango", "Orange"]`. The target keyword is `"Flower"`. The binary search function is called, and the result is printed. The output shows that the keyword "Flower" was found at position 5. The code execution was successful.

```
18     else:
19         low = mid + 1
20
21     return -1
22
23
24 keywords = [
25     "Apple",
26     "Banana",
27     "Cat",
28     "Dog",
29     "Elephant",
30     "Flower",
31     "Mango",
32     "Orange"
33 ]
34
35
36 target = "Flower"
37
38 result = binary_search(keywords, target)
39
40 if result != -1:
41     print("Keyword found at position:", result)
42 else:
```

Output:

```
Keyword found at position: 5
=== Code Execution Successful ===
```

Complexity Evaluation:

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Space Complexity : $O(1)$

Conclusion:

Binary Search is an efficient algorithm for search engine query processing. By applying the divide-and-conquer technique, it improves search efficiency and reduces execution time. Therefore, Binary Search is an optimal solution for search engines.